

# Complete PDF RAG Project Guide

## Introduction

This document explains the complete working of a local PDF Retrieval-Augmented Generation (RAG) project built using Python, LangChain, Ollama and ChromaDB. It is intended to be indexed by the project itself so the assistant can answer questions about its own architecture and implementation.

## End-to-End Workflow

1. User places PDF books in the books folder.
2. The application scans every PDF.
3. PyPDFLoader extracts text from each page.
4. RecursiveCharacterTextSplitter divides pages into overlapping chunks.
5. Each chunk becomes a LangChain Document with metadata such as source file and page number.
6. OllamaEmbeddings converts every chunk into a numerical embedding vector using mxlbai-embed-large.
7. ChromaDB stores vectors and metadata persistently.
8. When the user asks a question, the same embedding model converts the question into a vector.
9. Chroma performs similarity search and retrieves the top matching chunks.
10. The retrieved context is inserted into a prompt template.
11. Llama 3.2 receives the prompt and generates an answer grounded in the retrieved text.

## Core Concepts

**LLM:** A Large Language Model predicts the next token and generates natural language responses.

**RAG:** Retrieval-Augmented Generation combines external knowledge with an LLM to reduce hallucinations.

**Embedding:** A dense numeric vector representing semantic meaning.

**Vector Database:** Stores embeddings and performs nearest-neighbour search.

**Retriever:** Component responsible for finding the most relevant chunks.

**Prompt Template:** Combines user question and retrieved context before sending to the LLM.

## Project Files

**main.py** orchestrates the application, accepts user questions, retrieves relevant chunks and invokes the language model.

**vector.py** loads PDFs, creates chunks, generates embeddings and initializes Chroma.

**books/** contains all PDF files.

**chroma\_db/** stores the vector index.

**requirements.txt** lists project dependencies.

## Chunking

Chunking is essential because LLMs have limited context windows. Overlapping chunks preserve continuity across page boundaries and improve retrieval accuracy.

## Metadata

Every chunk should include metadata such as source filename, page number, chunk number and optionally chapter title. Metadata allows answers to cite their origin.

## Similarity Search

The user's question is embedded using the same embedding model. Chroma compares this vector with stored vectors using vector similarity and returns the top-k closest chunks.

## **Prompt Construction**

The retrieved chunks are concatenated into a context block. The prompt instructs the LLM to answer only from this context and admit when the answer is unavailable.

## **Advantages**

Runs locally, no API keys, privacy-friendly, reusable knowledge base, scalable to many books, supports semantic search.

## **Limitations**

Scanned PDFs require OCR. Embedding quality affects retrieval. Very large libraries benefit from incremental indexing and metadata management.

## **Future Improvements**

Incremental indexing, SHA-256 file hashing, automatic synchronization, OCR support, citations, reranking, hybrid search, conversational memory, Streamlit interface, multi-format document ingestion (PDF, DOCX, HTML, Markdown, TXT).

## **Interview Topics**

Understand LangChain, Documents, Embeddings, ChromaDB, Vector Search, RAG architecture, Chunking, Metadata, Prompt Engineering, Ollama, Local LLMs, Similarity Search, and production indexing strategies.

## **Summary**

The project implements a complete local RAG pipeline: document ingestion → chunking → embeddings → vector database → retrieval → prompt construction → LLM generation. This architecture enables accurate question answering over private PDF collections without relying on cloud AI services.

**Project By: Shubham Singh**